

ChunkNorris: A High-Performance and Low-Energy Approach to PDF Parsing and Chunking

Mathieu Ciancone
Wikiti / Lyon, France
mathieu@wikiti.ai

Clovis Varangot-Reille
Wikiti / Lyon, France
Laboratoire Hubert Curien
Université Jean Monnet
Saint-Etienne, France
clovis@wikiti.ai

Marion Schaeffer
Wikiti / Lyon, France
INSA Rouen Normandie
Rouen, France
marion.schaeffer
@insa-rouen.fr

Abstract

In Retrieval-Augmented Generation applications, the Information Retrieval part is central as it provides the contextual information that enables a Large Language Model to generate an appropriate and truthful response. High quality parsing and chunking are critical as efficient data segmentation directly impacts downstream tasks, i.e. Information Retrieval and answer generation. In this paper, we introduce ChunkNorris, a novel heuristic-based technique designed to optimise the parsing and chunking of PDF documents. Our approach does not rely on machine learning and employs a suite of simple yet effective heuristics to achieve high performance with minimal computational overhead. We demonstrate the efficiency of ChunkNorris through a comprehensive benchmark against existing parsing and chunking methods, evaluating criteria such as execution time, energy consumption, and retrieval accuracy. We propose an open-access dataset to produce our results. ChunkNorris outperforms baseline and more advanced techniques, offering a practical and efficient alternative for Information Retrieval tasks. Therefore, this research highlights the potential of heuristic-based methods for real-world, resource-constrained RAG use cases. The package is available at <https://github.com/wikiti-ai/chunknorris>.

1 Introduction

Retrieval-Augmented Generation (RAG) is an advanced paradigm in Natural Language Processing (NLP) that combines the strengths of Information Retrieval (IR) and generative models to address tasks requiring extensive knowledge and contextual understanding (Lewis et al., 2020). Unlike standalone generative models, RAG dynamically integrates external knowledge sources by retrieving relevant documents or data during inference¹.

¹<https://arxiv.org/pdf/2312.10997>, accessed on February 3, 2026.

This retrieval step ensures the generated responses are coherent as well as grounded in up-to-date and accurate information, mitigating issues like hallucination (Bouvard et al., 2024). Therefore, RAG is particularly valuable for applications such as question-answering, conversational agents, document summarisation, and decision support (Fan et al., 2024).

The retrieval step in an RAG system queries a knowledge base, typically a large corpus of unstructured or semi-structured documents, to identify and extract the most relevant content for a given input (Bouvard et al., 2024). This process often involves techniques from the field of IR, such as lexical matching (e.g., TF-IDF (Sparck Jones, 1988), BM25 (Robertson and Jones, 1976)) or dense vector retrieval using embeddings generated by pre-trained language models (e.g., Sentence-BERT (Reimers and Gurevych, 2019)). The retrieved results are ranked based on their relevance to the query and passed to the generative model as context (Tao et al., 2023). Advanced systems may incorporate hybrid retrieval strategies, combining heuristic-based approaches with Machine Learning (ML) for improved performance.

To feed the retriever of a RAG system, a knowledge base is constructed from documents that vary greatly in terms of format and complexity (Zhang et al., 2024). To ensure good retrieval performance, documents must be parsed and chunked. Regarding PDF documents, parsing involves extracting structured and unstructured data from a format designed for human readability rather than machine processing, often requiring the handling of intricate layouts, multi-column text, tables, figures, and metadata². The process typically begins with text extraction using libraries or tools such as *PyPDF2*³,

² <https://arxiv.org/abs/2410.09871v1>, accessed on February 3, 2026.

³ <https://pypdf2.readthedocs.io/en/3.x/>, accessed on February 3, 2026.

*PDFPlumber*⁴, or *PyMuPDF*⁵, followed by additional preprocessing to clean and structure the extracted content. Key challenges include preserving semantic coherence, managing irregular formatting, and accurately reconstructing the document’s logical flow².

Once the document structure and raw text are extracted from a document through parsing techniques, that content undergoes chunking which aims to segment the content into smaller, semantically coherent units, or *chunks*, to facilitate efficient storage, indexing, and retrieval (Kshirsagar, 2024). This segmentation ensures that the retrieval system can identify and provide precise, contextually relevant information in response to a query rather than retrieving entire documents or unmanageable text blocks⁶. Effective chunking strategies balance granularity, ensuring chunks are neither too large to dilute relevance nor too small to lose context⁶. Techniques for chunking often leverage heuristic rules, such as splitting by paragraph, sentence, or headings, while more advanced methods may incorporate semantic analysis to group related content meaningfully (Kshirsagar, 2024).

Therefore, we aim to propose an efficient and low-energy solution for parsing and chunking PDF documents to improve IR performance: ChunkNorris. We benchmark ChunkNorris with existing parsing and chunking techniques on various criteria with an open-access dataset we build and propose to the community. The code for the benchmark is available in our GitHub repository: <https://github.com/wikit-ai/bench-chunk-norris-acl2025>

2 Related Work

The PDF was invented to address the need to encapsulate documents to ensure readability across various platforms. Since its inception, continuous research has been conducted on how to effectively parse PDF files, given their complex structure. With the rise of Large Language Models (LLMs) and RAG applications, the need for efficient and accurate PDF parsing has become more critical. Significant advancements have been made in parsing techniques for PDF documents, driven by the increasing need to extract structured data

from inherently unstructured or semi-structured content. State-of-the-art methods often combine traditional approaches with ML techniques to handle the complexities of the PDF format. Traditional methods rely on libraries like *PDFMiner*⁷ or *PyPDF2*³, which provide programmatic access to text, images, and metadata. However, these tools only provide simple text extraction and do not provide information about document layout, hierarchical information, or structured information like tables. Heuristics-based methods were initially developed to enhance basic text extraction. However, they tend to be replaced in favour of computer vision approaches that leverage ML to improve performance.

Models trained with annotated data integrate textual and layout information, enabling improved tables, forms, and complex structures extraction. This is the case for Python libraries such as *Unstructured*⁸ and *Docling*⁹. Furthermore, hybrid techniques combine multiple approaches, such as *Open-Parse*¹⁰, which uses mainly heuristics, and ML as an option; *Marker*¹¹, which uses ML, Optical Character Recognition (OCR), and has LLM support; and *LLM Sherpa*¹², which uses heuristics and LLM techniques. While such techniques provide high-quality results, their use in production is limited due to high processing time, computational resources and annotation requirements that often do not match the constraints of production environments.

Regarding chunking, the literature suggests two possible scenarios. The first is where the parsing and chunking methods are separate. For example, *PyPDF2*, *Unstructured*, *Marker*, *LLM Sherpa* and *NV-Ingest*¹³ are parsing-only tools. Their output can be processed using independent chunking techniques, such as the *LangChain*’s commonly used *recursive character text splitter*¹⁴. In the second

⁴<https://github.com/jsvine/pdfplumber>, accessed on February 3, 2026.

⁵<https://pymupdf.readthedocs.io/en/latest/>, accessed on February 3, 2026.

⁶<https://arxiv.org/html/2402.05131v3>

⁷<https://pdfminersix.readthedocs.io/en/latest/>, accessed on February 3, 2026.

⁸<https://docs.unstructured.io/welcome>, accessed on February 3, 2026.

⁹<https://ds4sd.github.io/docling/>, accessed on February 3, 2026.

¹⁰<https://github.com/Filimoa/open-parse>, accessed on February 3, 2026.

¹¹<https://github.com/VikParuchuri/marker>, accessed on February 3, 2026.

¹²<https://github.com/nlmatrics/llmsherpa>, accessed on February 3, 2026.

¹³<https://github.com/NVIDIA/nv-ingest>, accessed on February 3, 2026.

¹⁴https://python.langchain.com/docs/how_to/recursive_text_splitter/, accessed on February 3, 2026.

case, the parsing and chunking methods are built together as a pipeline to combine these two steps effectively. This is the case, for example, for *Open-Parse* and *Docling*. Among the most common chunking methods is length-based splitting, which segments text based on specified size limits, such as tokens or characters. Token-based splitting is particularly useful when interfacing with language models, as it aligns with their input constraints, while character-based splitting ensures consistency across diverse text types (Kshirsagar, 2024). These methods are straightforward to implement, adaptable, and produce uniform chunk sizes. A more advanced approach is hierarchical splitting, which leverages the natural structure of text, such as paragraphs, sentences, and words, to create coherent splits (Kshirsagar, 2024). Tools like LangChain’s recursive character text splitter¹⁵ exemplify this technique by prioritising larger units (e.g., paragraphs) and recursively splitting smaller units when necessary, preserving the semantic flow of the text. For documents with inherent structures, such as HTML, Markdown, or JSON, structure-based splitting leverages these formats’ structural information, such as headers, tags, or object boundaries, to create contextually rich chunks (Kshirsagar, 2024). This approach maintains the document’s logical organisation and is particularly effective for preserving semantic relationships. Finally, semantic-based splitting goes a step further by analysing the content’s meaning to identify significant shifts in context (Kshirsagar, 2024). This last method often uses sliding window techniques and embeddings to detect breakpoints in the text, ensuring chunks remain semantically coherent. While semantic-based splitting stands out for its ability to directly analyse and maintain contextual integrity, it requires much more execution time and computational resources that are not always available in practice.

With ChunkNorris, we propose an efficient and unsupervised ML-free parsing and chunking method. We aim to ensure fast document ingestion with limited computational resources while getting the most out of the document structure. Our algorithm is robust and enables coherent chunking of various documents.

¹⁵https://python.langchain.com/docs/how_to/recursive_text_splitter/, accessed on February 3, 2026.

3 ChunkNorris

This work introduces ChunkNorris, a novel parsing and chunking algorithm designed to efficiently process documents without requiring GPU acceleration. While chunking based on document titles has demonstrated high effectiveness, it has not been widely applied to PDFs, primarily because detecting headers and their hierarchy from document layout is challenging. We developed ChunkNorris, which leverages title-based segmentation to produce high-quality chunks while remaining lightweight and efficient.

3.1 Parser

A parser is a computational tool or algorithm that analyses and processes structured or unstructured data, converting it into a machine-readable format. Parsers are essential components of text processing systems that clean and format input documents. ChunkNorris parser’s primary role is to take a file or a string as input and produce a clean, markdown-formatted output suitable for further processing by a chunker. Currently, ChunkNorris supports three parsers: MarkdownParser, HTMLParser, and PdfParser. Regardless of the input type, all parsers generate a unified MarkdownDoc object, which serves as input for a chunker. This work focuses on the PdfParser, which is designed to extract and structure content from PDF files.

ChunkNorris implementation relies on the *PyMuPDF* library. This tool provides fast implementations of a great variety of utility functions for document processing. The parsing process begins by opening the PDF using *PyMuPDF*. Text is retrieved in the form of spans, which are defined as sets of consecutive characters sharing the same formatting properties. Based on their attributes and location, spans undergo a series of processing steps to ensure accurate structuring of the document.

Headers and footers: If a span’s bounding box appears in the same position on more than 33% of the document’s pages, it is flagged as a header or footer and subsequently removed.

Links: In PDF documents, hyperlinks exist as invisible clickable boxes layered over spans. The PdfParser retrieves links and binds them to their corresponding spans to avoid losing that information. To our knowledge, none of the other existing PDF parsing tools extract hyperlinks.

Tables: Tables in PDFs vary widely in layout, making them challenging to parse. They fall into three

categories: those with visible cell boundaries, those inferred from content alignment, and those embed as images. In the first case, the table structure is represented by line vectors, which can be recombined for parsing. ChunkNorris employs a vectorised line recombination method for efficient table structure extraction. It also handles the parsing of tables with merged cells.

Lines/Blocks: Following text extraction, spans are grouped to form coherent text units. Consecutive spans on the same vertical position are merged into lines. Lines are further grouped into blocks, which may represent a paragraph or a section title. Building of blocks is based on the line spacing of the document’s body content, which refers to the vertical distance between bounding boxes of consecutive lines.

Main title: The parser also attempts to infer the document’s main title by analysing blocks on the first page. Blocks with font sizes larger than the body text are considered potential title candidates.

Section headers: The PdfParser detects section headers and their hierarchy. It first checks the document metadata for a Table of Contents (ToC). If found, it is used directly. Otherwise, the parser searches for a structured ToC within the document using regular expressions. Header levels are inferred from indentation (deeper levels are further right) or numbering patterns (e.g., *I.*, *I.1*, *I.1.a*). If no ToC is available, font sizes determine hierarchy, with smaller fonts indicating deeper levels.

Other: Various document attributes are also extracted to characterise the content, such as document orientation, font size of the body, etc.

Finally, after completing the extraction and structuring process, the PdfParser generates a markdown-formatted document, ensuring a clean and structured representation of the original PDF content ready to be processed by a chunker or other text-processing modules.

3.2 Chunker

Chunkers process parser outputs by segmenting them into coherent units. In ChunkNorris, all parsers produce Markdown-formatted Markdown-Doc objects, ensuring compatibility with the MarkdownChunker. Markdown is the chosen standard for its readability by both humans and LLMs while offering sufficient structure for chunking. As a result, the MarkdownChunker is currently the only implemented chunker, though others may be developed if new parser output formats emerge.

The chunking strategy employed by the MarkdownChunker is based on several guiding principles. First, each chunk must contain homogeneous information. Therefore, the chunking process relies on document section headers to define chunk boundaries. Second, each chunk must retain contextual information, as sections of a document can lose meaning when read in isolation. To preserve context, the headers of all parent sections are prepended to each chunk. Third, chunk sizes should be as uniform as possible. Embedding models used in IR are sensitive to chunk length, resulting in higher embedding similarity for chunks of similar length to the query. If a chunk is significantly longer than the query, its similarity score may decrease despite relevance. Chunkers aim to maintain a consistent chunk size whenever possible to mitigate this issue.

The chunking process begins by constructing a ToC tree from document headers. Chunks are then recursively generated based on the ToC structure, with each chunk containing the titles of upper sections and the content of the current section. A chunk is subdivided using available subsections if it exceeds the soft word limit. Otherwise, it remains intact. After chunking, refinements are applied: sections below a minimum word count are discarded to ensure only meaningful chunks are retained. Chunks exceeding a hard limit are split into subchunks at newline characters, which ensure tables and code blocks remain within a single chunk. Titles from the original chunks are preserved at the beginning of each subchunk to maintain context.

The final output of the MarkdownChunker is a list of Chunk objects, each containing its processed text, the associated parent headers, the starting line of the chunk within the original markdown-formatted document and, when relevant, the start and end page numbers of the paginated source file. This structured output ensures very fast and efficient document segmentation while preserving readability and contextual integrity. ChunkNorris parsers and chunkers can be wrapped up into pre-built pipelines. They allow processing of documents with minimum code while ensuring constant output quality.

4 Benchmark of parsing and chunking techniques

To evaluate ChunkNorris, we propose comparing its performance to other popular tools. This section

presents the methodology we apply to compare ChunkNorris and the dataset constructed for the evaluation. The dataset is available as open-source on Hugging Face: <https://huggingface.co/datasets/Wikit/PIRE>.

4.1 Dataset

We evaluate parsing and chunking in the context of RAG. Therefore, we construct the PDF dataset for Information Retrieval Evaluation (PIRE) designed explicitly for the IR use case to assess the parsing and chunking methods. This dataset comprises 100 PDFs, combining 50 documents from the existing DocLayNet dataset (Pfitzmann et al., 2022) and 50 newly collected PDFs whose diversity reflects real-world use cases. The newly collected set includes 5 arXiv papers, 2 financial reports, 4 infographics, 4 legal documents, 3 IT documentations, 4 news articles, 3 PowerPoint-like documents, 13 PubMed papers, 3 organisation reports, 5 user manuals, and 5 Wikipedia articles. The documents were selected with consideration of their licenses. A list of all the PDFs and their references can be found in the Appendix A. This diverse selection ensures a broad range of document structures and content types, making it well-suited for evaluating the robustness of our approach.

Single-chunk dataset

We first annotate three questions per PDF, resulting in 300 question-document pairs. Three annotators followed a structured methodology to identify the minimal passage within each document that contained the answer. A retrieval step was carried out to ensure only one passage contained the answer to each question. In this step, documents were parsed using the ChunkNorris parser and segmented into fixed-size chunks of 250 words. We then embed all the chunks using *Alibaba-NLP/gte-large-en-v1.5*¹⁶, a high-performing embedding model from the MTEB leaderboard (Muennighoff et al., 2023) that remains computationally efficient with fewer than 500M parameters. We compute the cosine similarity between the annotated question and all chunks, retrieving the top 10 highest-scoring chunks. The annotators validate the question if no additional relevant chunks are found among these top results. Otherwise, they refine or rewrite the

question to better isolate a single relevant passage. We refer to this subset as *single-chunk dataset*.

Multi-chunk dataset

Additionally, we extend the dataset with another 32 questions, which require several pieces of information spread over multiple pages to be answered. This time, each question is matched with the pertinent passages of the corpus along with their source document and page. This part of the dataset is deliberately more complex than the first. We refer to it as *multi-chunk dataset*.

This dataset provides a high-quality benchmark for evaluating parsing and chunking strategies in an IR context. Incorporating diverse document types, structured annotations, and a robust validation process allows for a comprehensive assessment of our approach’s in real-world retrieval scenarios.

4.2 Evaluation methodology

To evaluate ChunkNorris, we compare its performance with other tools in the literature. We choose methods for parsing and/or chunking PDF documents to carry out our benchmark. After a wide-range screening of existing methods, we select those that show remarkable performance and have aroused great interest in the community. For the parsing step, we compare *Marker*, *Open-Parse* and *Docling*. *PyPDF* is used as a baseline, as it can only perform text extraction without properly parsing the document. All parsers are run with their default configuration. OCR is deactivated to avoid influencing the results, as it is not needed for the studied documents. We associate all these tools with two different chunking strategies: by page and with the recursive text splitter set to a size limit of 4000 characters per chunk and an overlap of 200 characters between chunks. Custom chunking strategies are available for *Open-Parse* and *Docling*, so we add them to the comparison. Additionally, *Open-Parse* proposes two parsing backends: one using *PyMuPDF*’s built-in functions and another leveraging Unitable (Peng et al., 2024), an ML framework for table extraction. The first one runs on CPU only while the latter demands a GPU. In the rest of this paper, they will be referred to as *Open-Parse-P* and *Open-Parse-U*, respectively. Appendix B summarises the features of the compared parsers.

We evaluate the pipelines based on various criteria, including execution time and environmental

¹⁶<https://huggingface.co/Alibaba-NLP/gte-large-en-v1.5>, accessed on February 3, 2026.